

반응형 포트폴리오 웹사이트 구축기

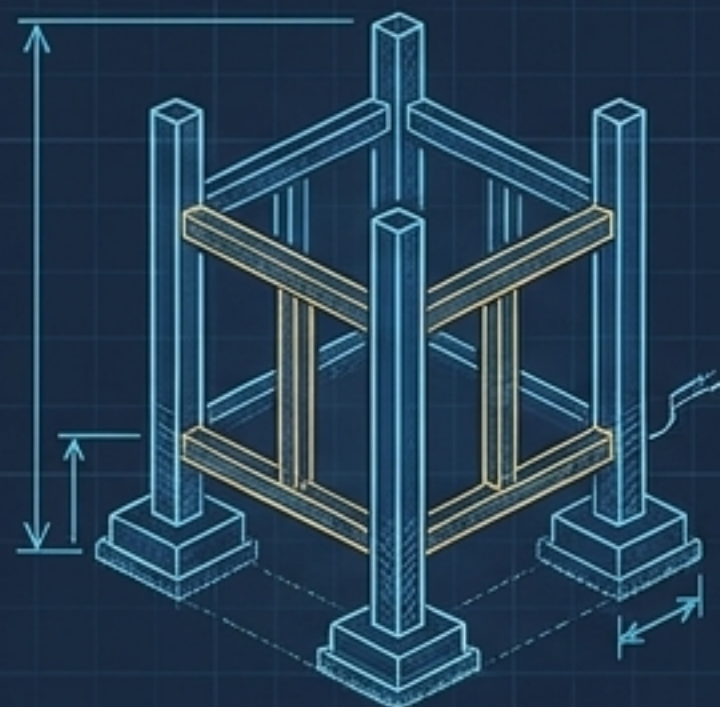
바닐라 JS로 이해하는 웹 프론트엔드의 기초 동작 원리

프레임워크 없이 순수 HTML/CSS/JS 만으로 집을 짓다



프레임워크 이전에 '기초 체력'을 다지는 시간

<HTML>



뼈대

(기둥을 세우고 방의 위치를 잡는 공사)

웹의 구조와 의미(Semantic Structure)



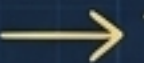
{CSS}



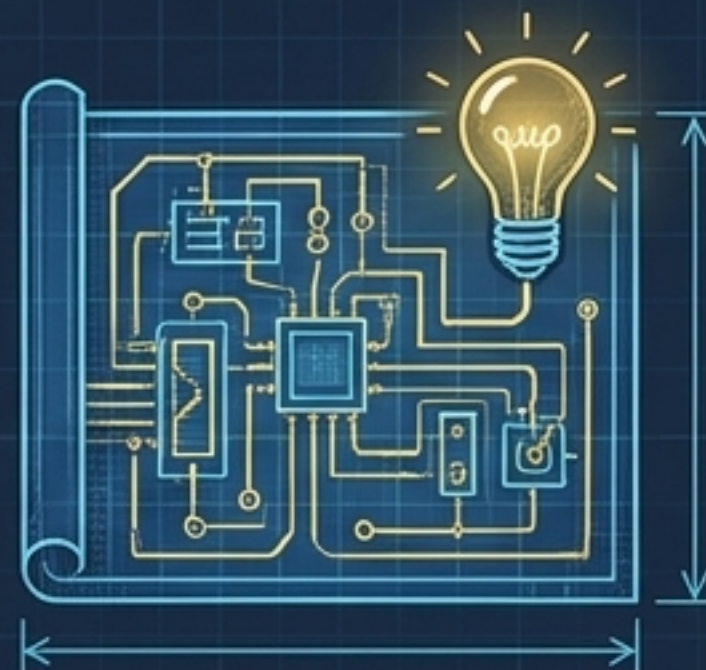
인테리어

(벽지를 바르고 조명을 꾸미는 작업)

시각적 스타일, 반응형 레이아웃, 테마



(JS)



전기 시스템

(스위치를 누르면 불이 켜지는 동적 기능)

사용자 이벤트 처리, 비동기 통신, DOM 조작

모든 최신 프레임워크는 결국 이 3가지 언어로 변환되어 동작합니다. 본질을 알면 도구에 휘둘리지 않습니다.

웹 구축을 위한 6대 핵심 공정 (Roadmap)



[과제 1] 시맨틱 HTML: 이름표가 붙은 상자들

<div> 남용을 멈추고 영역의 역할이 명확한 태그를 사용해야 합니다.



검색 엔진 최적화(SEO)

검색 엔진이 페이지의 구조와 핵심 내용을 더 잘 이해할 수 있어요.



접근성 향상

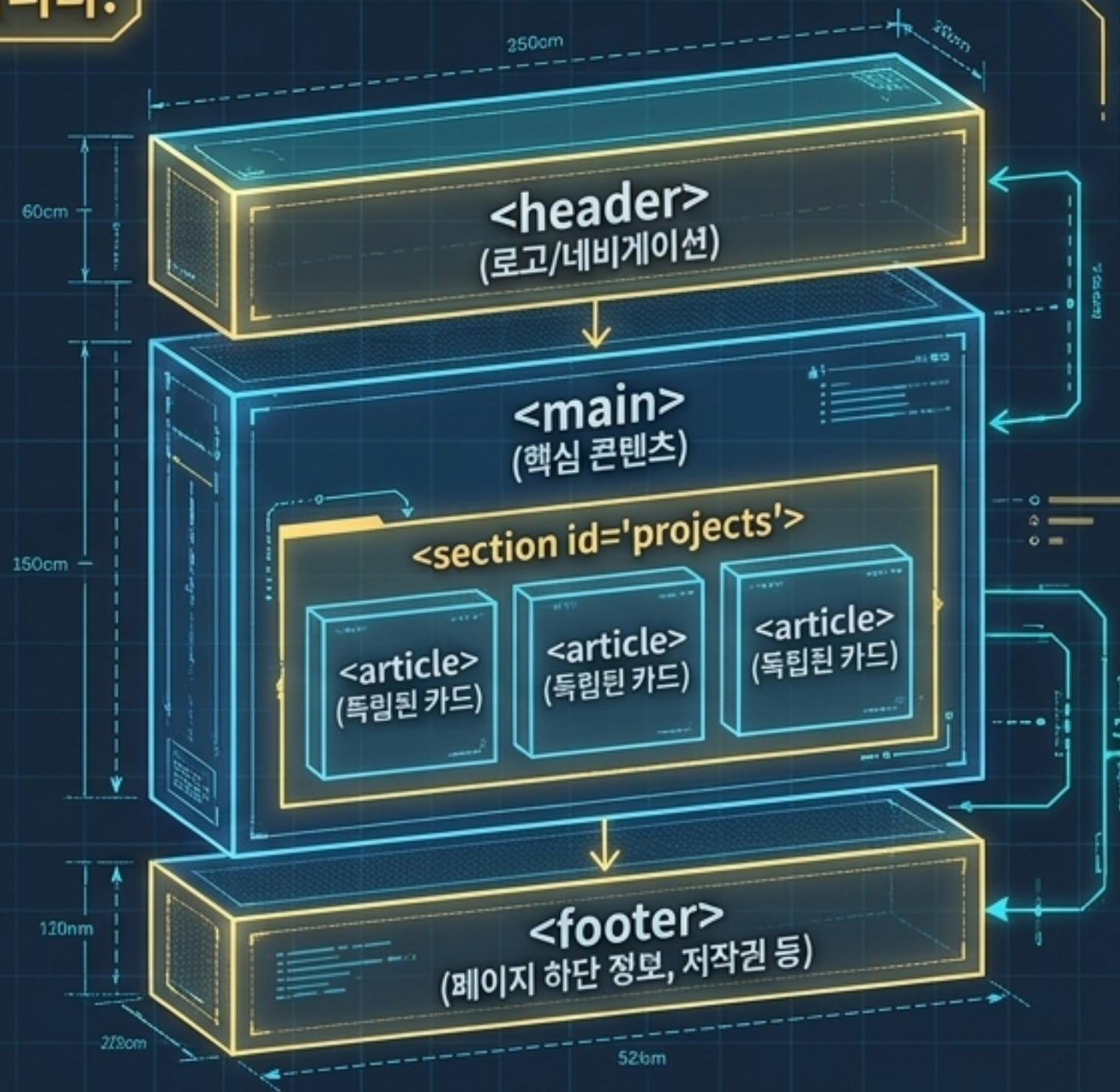
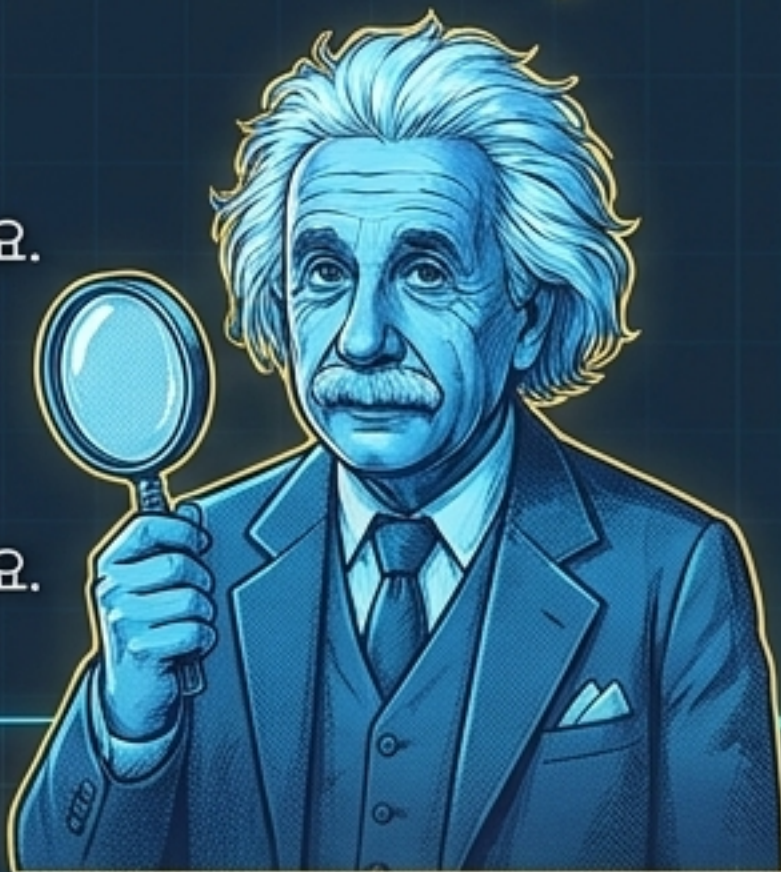
스크린 리더 같은 보조 기술이 콘텐츠를 더 정확하게 읽을 수 있어요.



유지보수의 용이성

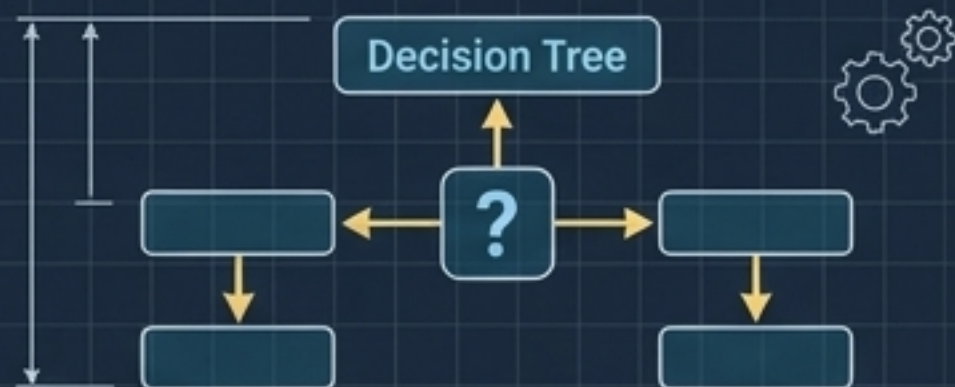
코드의 구조와 의미가 명확해서 다른 개발자도 쉽게 이해할 수 있어요.

웹을 더 똑똑하게 만듭니다!

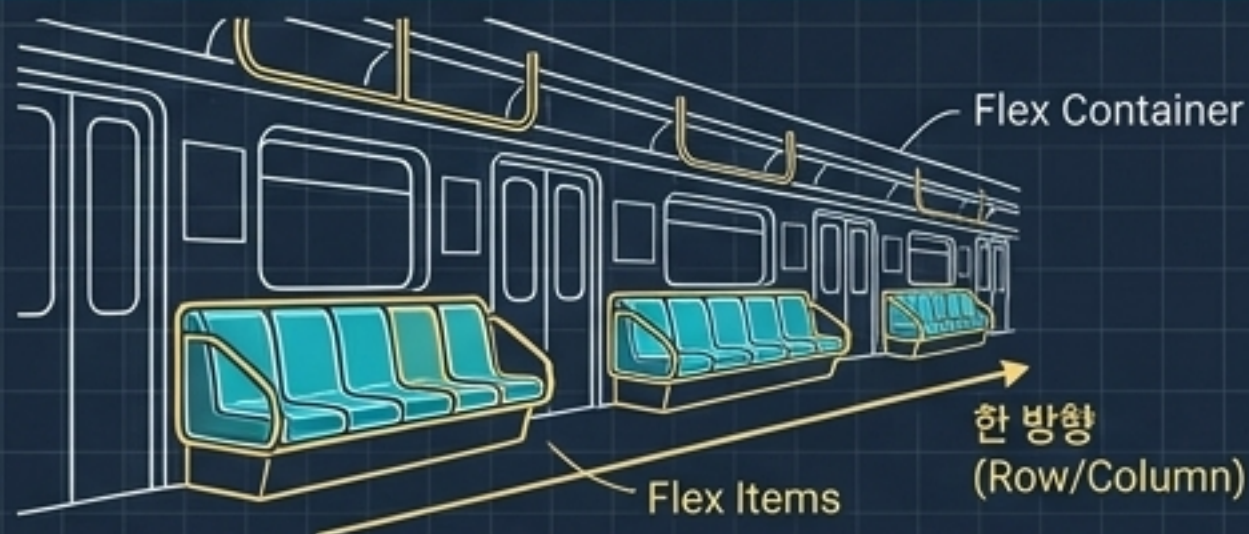


[과제 2] CSS 레이아웃: 언제 무엇을 쓸 것인가?

한 줄 정렬은 Flexbox, 바둑판 배열은 Grid를 선택합니다.



Flexbox (1차원 레이아웃)

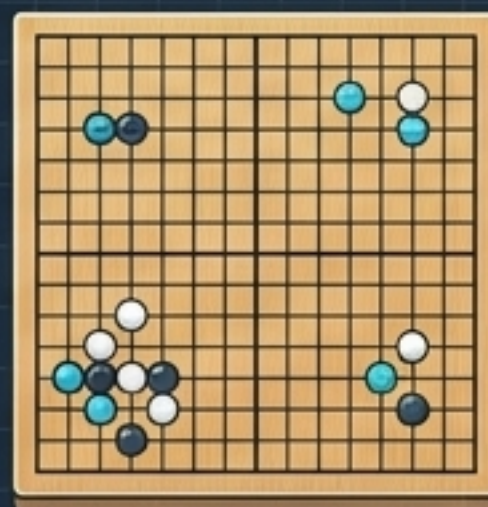


비유: 지하철 좌석 (가로 또는 세로 한 방향 정렬)

적용: 네비게이션 바 (<nav>)

```
1 .nav-container {  
2   display: flex;  
3   justify-content: space-between;  
4 }
```

Grid (2차원 레이아웃)



비유: 바둑판 (행과 열을 동시에 제어)

적용: 프로젝트 카드 목록 (.projects-grid)

```
1 .projects-grid {  
2   display: grid;  
3   /* 미디어 쿼리 없는 자동 반응형의 미법 */  
4   grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));  
5 }
```



[과제 3] DOM 제어: 요소를 낚아채고 반응하기

JS가 웹페이지를 조작하는 3단계 로직: 찾기 → 모으기 → 등록



핵심 포인트!



Step 1. 찾기 (Select)

querySelector로 CSS 선택자(ID, Class)를 이용해 요소를 꼭 집어냅니다.

```
const btn = document.querySelector('.btn');
```

Step 2. 모으기 (Cache)

찾은 요소들을 `const elements = {}` 객체에 한 번만 저장하여 반복 탐색(성능 저하)을 방지합니다.

```
const elements = { btn: document.querySelector('.btn') };
```

Step 3. 등록 (Bind)

addEventListener를 통해 클릭, 스크롤 등의 사용자 이벤트가 발생했을 때 실행할 함수를 연결합니다.

❌ Bad

```
<button onclick='...'>  
(구조와 로직이 섞임)
```

```
<button onclick="handleClick()">클릭</button>
```

✅ Good

```
btn.addEventListener('click', ...)  
(구조와 로직 분리)
```

```
btn.addEventListener('click', handleClick);
```

[과제 4] 최신 JS 문법 (ES6+): 코드를 압축하는 마법

화살표 함수와 배열 메서드로 데이터를 직관적으로 가공합니다.

파이프라인 (map & filter)



Filter

전체 저장소 중 특정 언어(Javascript)만 걸러냄.

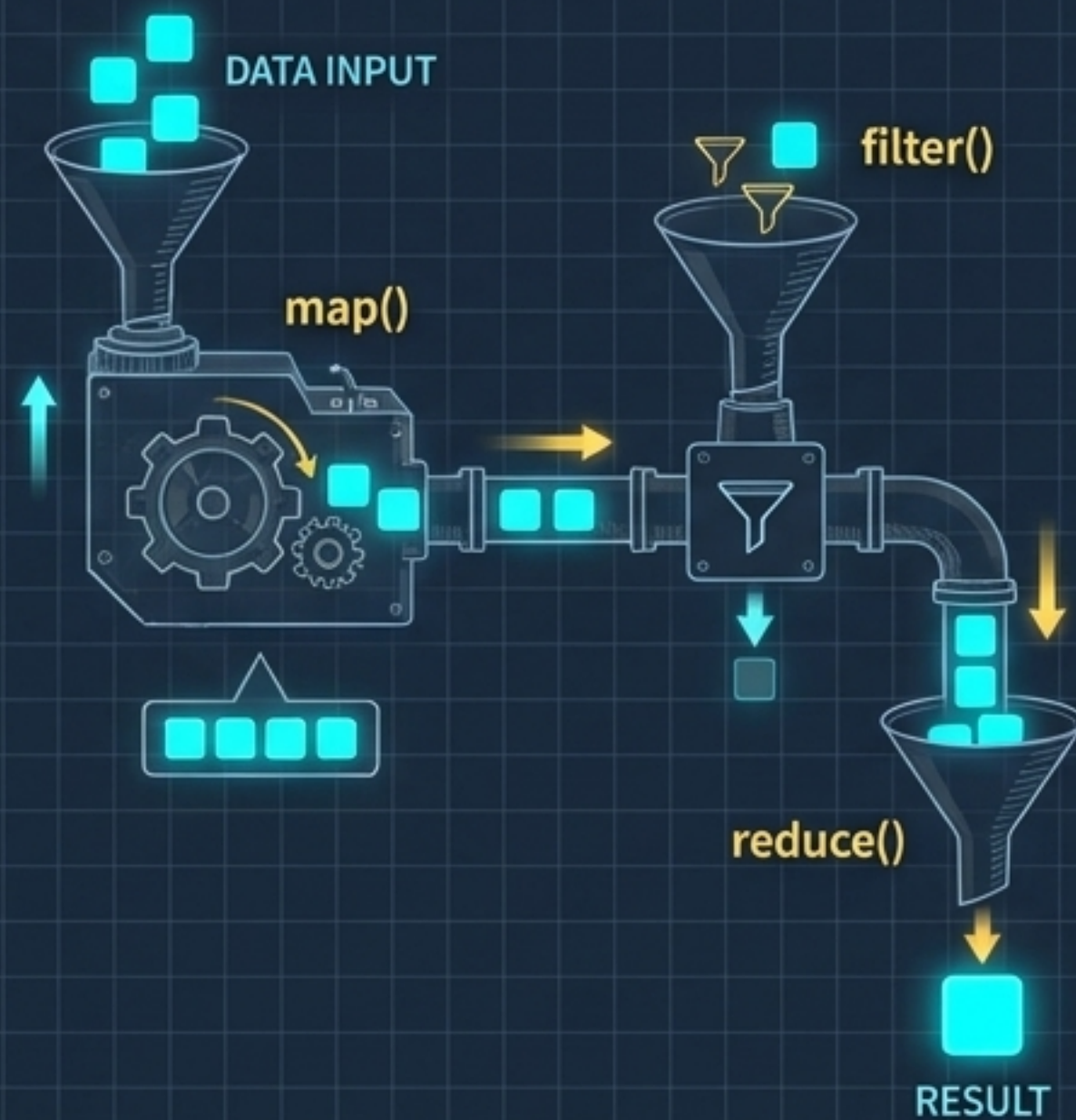
```
repos.filter(repo =>
  repo.language === 'JavaScript')
```



Map

걸러진 데이터를 HTML 문자열 카드(<article>)로 1:1 변환.

```
filteredRepos.map(repo =>
  `<article><h3>${repo.name}</h3>
</article>`)
```



구조분해 할당 (Destructuring)

Before X

```
1
2 // 복잡한 repo.name, repo.html_url
3 const name = repo.name;
4 const html_url = repo.html_url;
5
```

After ✓

```
1 const { name, html_url } = repo;
2
3
```

필요한 데이터만 쏙 빼내는 마법.
가독성이 극적으로 향상됩니다.



FRONTEND ENGINEERING
BLUEPRINT // JS ES6+

[과제 5-1] 비동기 통신: 데이터를 기다리는 우아한 방법

fetch와 **async/await**를 사용하여 커피 대기줄(UI 멈춤)을 **진동벨 시스템**(비동기)으로 바꿉니다.



Code Comparison (The Upgrade)

과거 (.then 방식)

콜백 체인으로 코드가 길어지고
가독성이 떨어짐.

현대 (async/await)

동기식 코드처럼 위에서
아래로 깔끔하게 읽힘.

```
const response = await fetch('https://api.github.com/...');  
const data = await response.json();
```

이 줄에서 응답이 올
때까지 기다린다.



안전망 장착: API 속도 제한(Rate Limit 403)이나
네트워크 오류에 대비한 **try...catch** 블록의 중요성.

[과제 5-2] 비동기 상태의 시각화 (UI States)

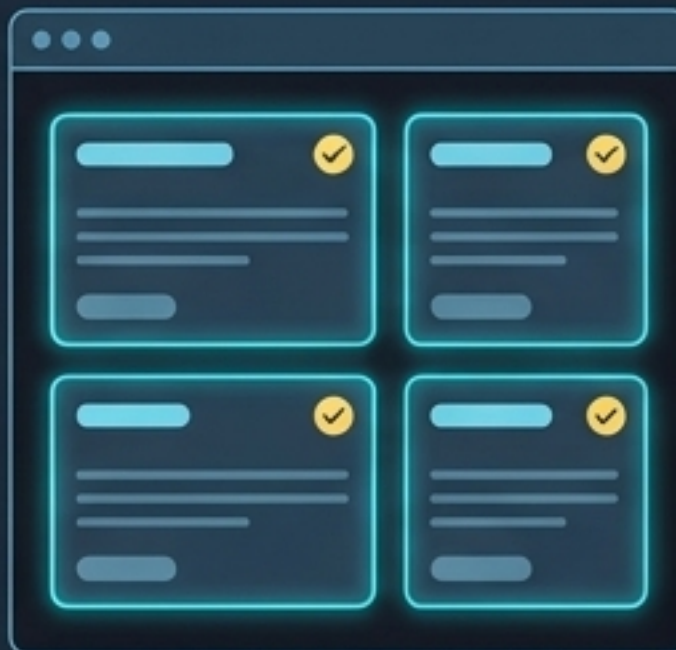
데이터를 기다리거나 실패했을 때, 사용자는 무엇을 보아야 하는가?

1. Loading (로딩 중)



메시지:
데이터를 가져오는 중입니다...
(사용자 불안감 해소)

2. Success (성공)



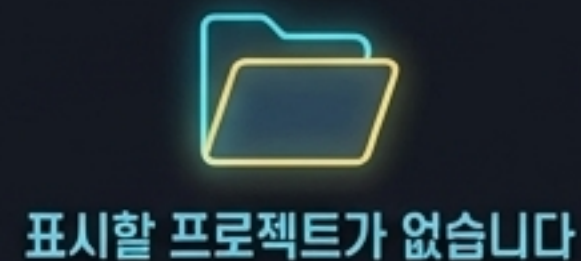
메시지: 정상 렌더링 완료.

3. Error (실패/오류)



대응: Rate Limit(403) 발생 시
즉각적인 피드백.

4. Empty (데이터 없음)



안내 메시지 렌더링.



FRONTEND ENGINEERING

BLUEPRINT // ASYNC UI STATES

[과제 6] 상태(State) 기반 렌더링

DOM을 직접 찌르지 마세요. 상태(State)를 바꾸면 화면은 알아서 따라옵니다.

Bad Pattern (직접 제어)

클릭 -> 메뉴 태그 찾기 -> style.left = 0 주입 -> 버튼 active 클래스 추가
(현재 상태 추적 불가)

Good Pattern (Single Source of Truth)



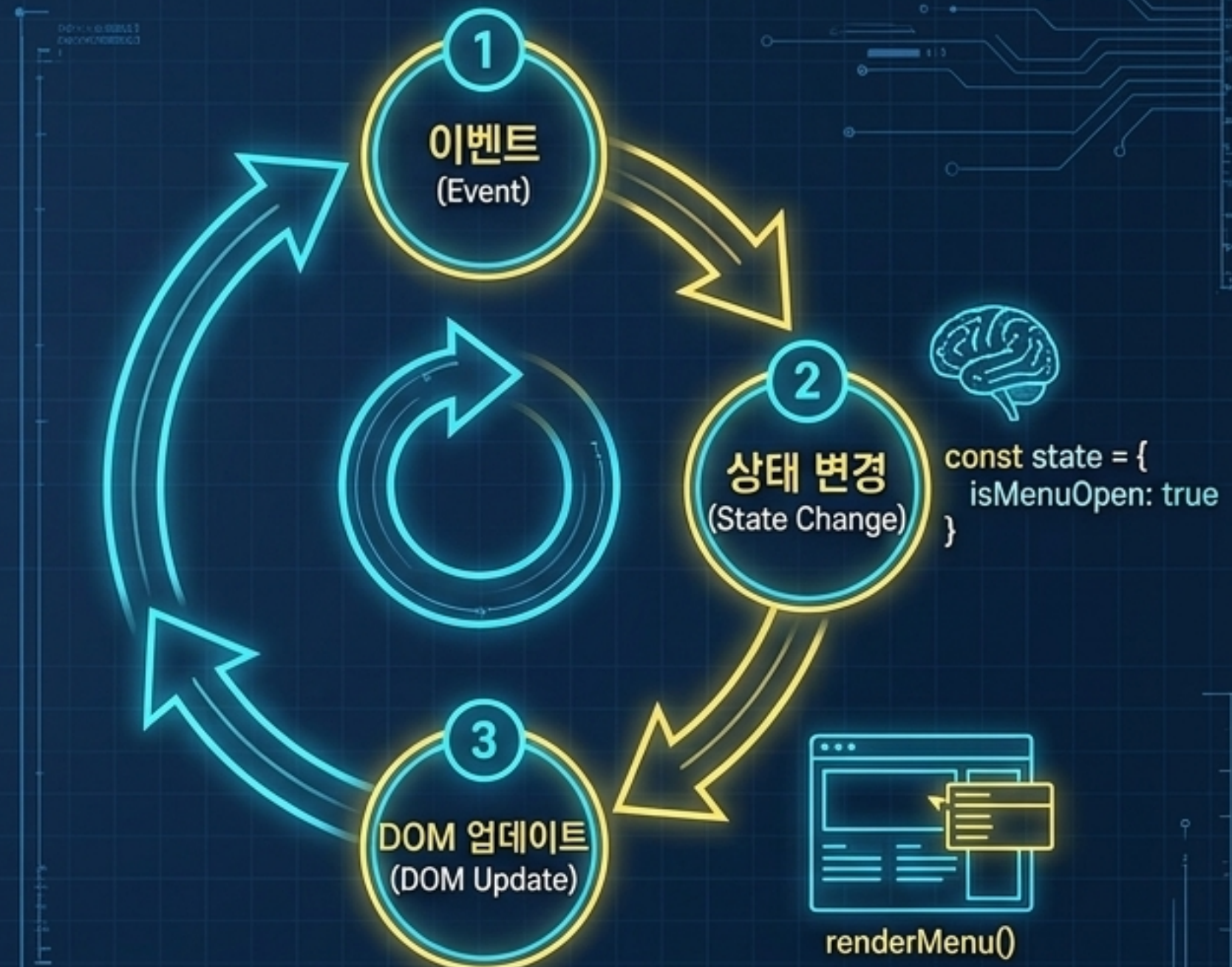
중앙 통제실:
`const state = {
 isOpen: false
}`

1단계 (Event):
클릭 시 `state.isOpen = true`로 상태만 변경.

2단계 (Render):
`renderMenu()` 함수가 상태를 확인하고 알아서 화면을 그림.



Insight: 이 패턴이 바로 현대 프론트엔드 프레임워크(React)가 동작하는 핵심 원리입니다.



[통합 적용 1] 다크 모드 전환의 해부학

버튼 클릭 한 번에 웹사이트 전체 색상이 바뀌는 내부 메커니즘

Event (클릭 감지)



```
btn.addEventListener('click', ...);
```

State (상태 업데이트 & 저장)



```
state.theme = 'dark';  
localStorage.setItem('theme', state.theme);
```

Render (화면 업데이트)



```
document.documentElement.setAttribute('data-theme', state.theme);
```



Tip: CSS의 `:root [data-theme='dark']` 변수 세트가 즉시 교체됨을 명시.



FRONTEND ENGINEERING

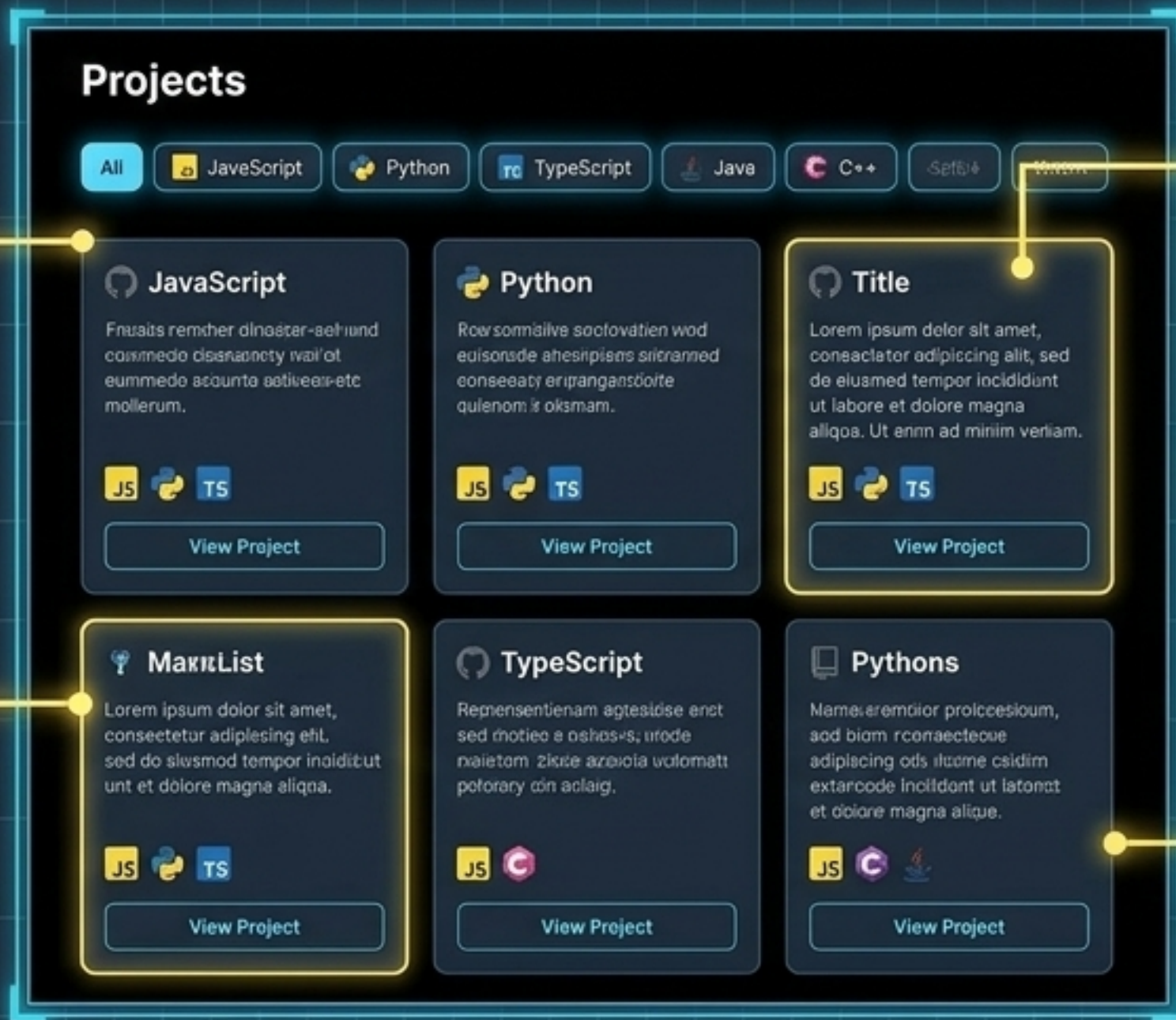
BLUEPRINT // DARK MODE ANATOMY

[통합 적용 2] 모든 기술의 결합: GitHub API 프로젝트 렌더링

6가지 핵심 기술이 모여 만들어진 완벽한 시너지

1 **카드 배열: Grid**
(2차원 자동 반응형 레이아웃)

2 **개별 카드 구조:**
시맨틱 `<article>` 태그



3 **카드 내부 데이터:**
ES6 구조분해 할당
& 템플릿 리터럴

4 **상단 언어 필터 버튼:**
배열 `filter()` 메서드
& 이벤트 위임

5 **데이터 로딩 프로세스:**
`fetch` & `async/await`
비동기 통신



FRONTEND ENGINEERING

BLUEPRINT // GITHUB API PROJECT INTEGRATION

결론: 프레임워크 너머의 '본질'을 보다

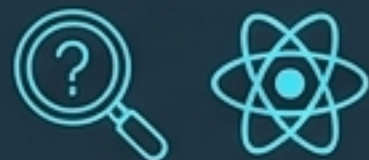
기초가 튼튼한 건물은 어떤 도구를 사용하든 무너지지 않습니다.



순수 바닐라 JS로 구현하며 DOM 조작의 비용과 렌더링 원리를 체득했습니다.



파편화된 변수 대신 중앙 집중식 상태(State) 관리의 강력함을 배웠습니다.



도구(React)의 '사용법'을 넘어, 도구가 만들어진 '이유(Why)'를 이해하게 되었습니다.

단단한 기초 위에서
사용자와 소통하는 개발자가 되겠습니다.

PROFILE



GitHub 저장소



포트폴리오 배포 URL

FRONTEND ENGINEERING

BLUEPRINT // CONCLUSION & CONTACT

Gemini Notebook